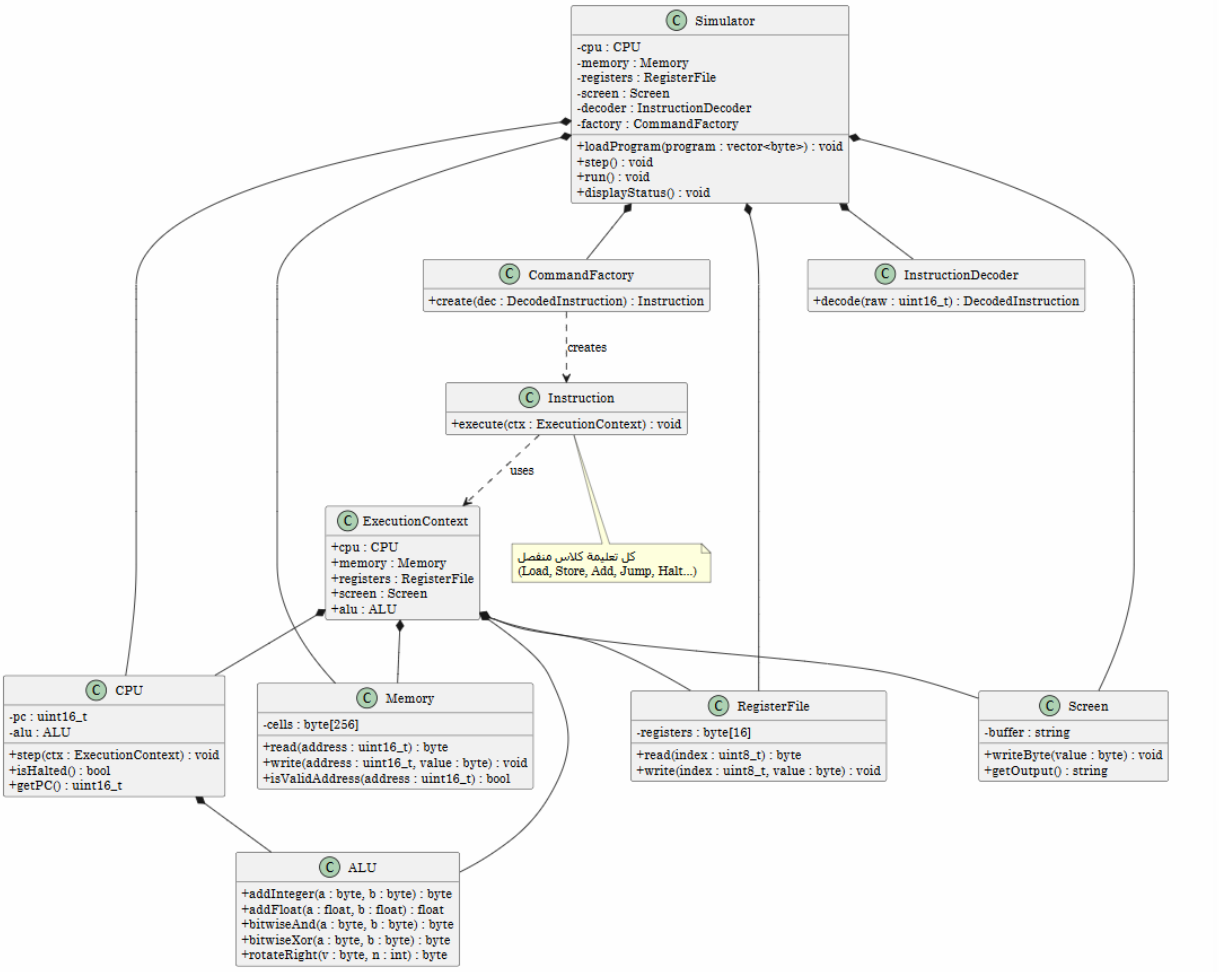
Final Design

UML Class Diagram



PlantUML Code

```
@startuml
skinparam classAttributeIconSize 0
skinparam packageStyle rectangle
hide empty members

class Simulator {
  - cpu : CPU
  - memory : Memory
  - registers : RegisterFile
  - screen : Screen
  - decoder : InstructionDecoder
  - factory : CommandFactory
  + loadProgram(program : vector<byte>) : void
  + step() : void
  + run() : void
  + displayStatus() : void
}

class CPU {
  - pc : uint16_t
  - alu : ALU
  + step(ctx : ExecutionContext) : void
  + isHalted() : bool
  + getPC() : uint16_t
}

class ALU {
  + addInteger(a : byte, b : byte) : byte
  + addFloat(a : float, b : float) : float
  + bitwiseAnd(a : byte, b : byte) : byte
  + bitwiseXor(a : byte, b : byte) : byte
  + rotateRight(v : byte, n : int) : byte
}

class Memory {
  - cells : byte[256]
  + read(address : uint16_t) : byte
  + write(address : uint16_t, value : byte) : void
  + isValidAddress(address : uint16_t) : bool
}

class RegisterFile {
  - registers : byte[16]
  + read(index : uint8_t) : byte
  + write(index : uint8_t, value : byte) : void
}

class Screen {
  - buffer : string
  + writeByte(value : byte) : void
  + getOutput() : string
}

class InstructionDecoder {
  + decode(raw : uint16_t) : DecodedInstruction
}

class Instruction {
  + execute(ctx : ExecutionContext) : void
}

class CommandFactory {
  + create(dec : DecodedInstruction) : Instruction
}
```

```
class ExecutionContext {
  + cpu : CPU
  + memory : Memory
  + registers : RegisterFile
  + screen : Screen
  + alu : ALU
}

Simulator *-- CPU
Simulator *-- Memory
Simulator *-- RegisterFile
Simulator *-- Screen
Simulator *-- InstructionDecoder
Simulator *-- CommandFactory
CPU *-- ALU
CommandFactory ..> Instruction : creates
Instruction ..> ExecutionContext : uses

ExecutionContext *-- CPU
ExecutionContext *-- Memory
ExecutionContext *-- RegisterFile
ExecutionContext *-- Screen
ExecutionContext *-- ALU

note bottom of Instruction
  كل تعليمة كلاس منفصل
  (Load, Store, Add, Jump, Halt...)
end note

@enduml
```